

Jacobian Descent For Multi-Objective Optimization

Valérian Rey — Simplex Lab
valerian.rey@gmail.com
Joint work with Pierre Quinton

February 11, 2026



github.com/TorchJD/torchjd



torchjd.org



arxiv.org/pdf/2406.16232

Table of contents

Theory

- Partial order

- Pareto optimality

- Jacobian descent

- Unconflicting projection of gradients (UPGrad)

Applications and experimentation

- Optimization trajectories

- Per-sample JD

- Multi-task learning

Conclusion

Theory

Partial order

Define the partial order $(\mathbb{R}^m, \preceq)$, for $\mathbf{u}, \mathbf{v} \in \mathbb{R}^m$, as

$$\begin{aligned} \mathbf{u} \preceq \mathbf{v} \\ \Leftrightarrow \quad u_i \leq v_i \quad \forall i \in [m] \end{aligned}$$

Partial order

Define the partial order $(\mathbb{R}^m, \preceq)$, for $\mathbf{u}, \mathbf{v} \in \mathbb{R}^m$, as

$$\begin{aligned} \mathbf{u} &\preceq \mathbf{v} \\ \Leftrightarrow \quad u_i &\leq v_i \quad \forall i \in [m] \end{aligned}$$

Examples:

$$\blacktriangleright \begin{bmatrix} 2 \\ 1 \end{bmatrix} \preceq \begin{bmatrix} 2 \\ 2 \end{bmatrix}$$

Partial order

Define the partial order $(\mathbb{R}^m, \preceq)$, for $\mathbf{u}, \mathbf{v} \in \mathbb{R}^m$, as

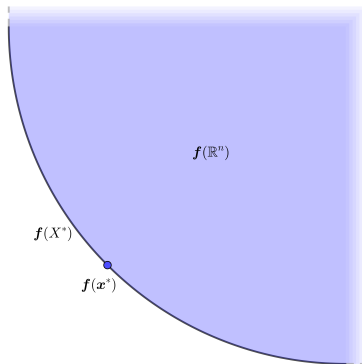
$$\begin{aligned} \mathbf{u} \preceq \mathbf{v} \\ \Leftrightarrow u_i \leq v_i \quad \forall i \in [m] \end{aligned}$$

Examples:

▶ $\begin{bmatrix} 2 \\ 1 \end{bmatrix} \preceq \begin{bmatrix} 2 \\ 2 \end{bmatrix}$

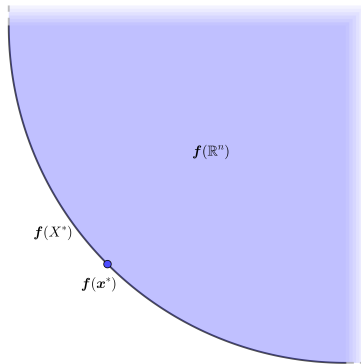
▶ $\begin{bmatrix} 2 \\ 1 \end{bmatrix}$ and $\begin{bmatrix} 1 \\ 2 \end{bmatrix}$ are not comparable with this partial order.

Pareto optimality



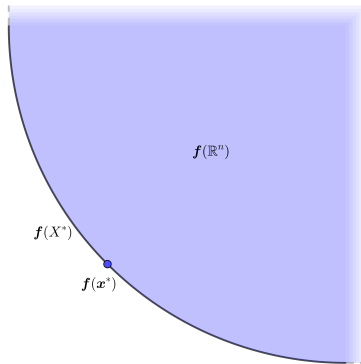
Pareto optimality

Pareto set: X^*



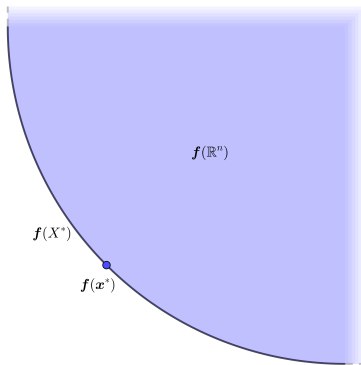
Pareto optimality

Pareto set: X^*
Pareto Front: $f(X^*)$



Pareto optimality

Pareto set: X^*
Pareto Front: $f(X^*)$



$$x^* \in X^* \quad \Leftrightarrow \quad \forall y \in \mathbb{R}^n, f(y) \preceq f(x^*) \rightarrow f(y) = f(x^*)$$

Jacobian descent

Goal: minimize $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ according to the $\preceq$ partial order.

Jacobian descent

Goal: minimize $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ according to the $\preceq$ partial order. For any $\mathbf{x} \in \mathbb{R}^n$, the Jacobian of $\mathbf{f}$ at $\mathbf{x}$, is

$$\mathcal{J}\mathbf{f}(\mathbf{x}) = \begin{bmatrix} \nabla f_1(\mathbf{x})^\top \\ \nabla f_2(\mathbf{x})^\top \\ \vdots \\ \nabla f_m(\mathbf{x})^\top \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x_1} f_1(\mathbf{x}) & \frac{\partial}{\partial x_2} f_1(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_1(\mathbf{x}) \\ \frac{\partial}{\partial x_1} f_2(\mathbf{x}) & \frac{\partial}{\partial x_2} f_2(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_2(\mathbf{x}) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} f_m(\mathbf{x}) & \frac{\partial}{\partial x_2} f_m(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_m(\mathbf{x}) \end{bmatrix}$$

Jacobian descent

Goal: minimize $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ according to the $\preceq$ partial order. For any $\mathbf{x} \in \mathbb{R}^n$, the Jacobian of $\mathbf{f}$ at $\mathbf{x}$, is

$$\mathcal{J}\mathbf{f}(\mathbf{x}) = \begin{bmatrix} \nabla f_1(\mathbf{x})^\top \\ \nabla f_2(\mathbf{x})^\top \\ \vdots \\ \nabla f_m(\mathbf{x})^\top \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x_1} f_1(\mathbf{x}) & \frac{\partial}{\partial x_2} f_1(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_1(\mathbf{x}) \\ \frac{\partial}{\partial x_1} f_2(\mathbf{x}) & \frac{\partial}{\partial x_2} f_2(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_2(\mathbf{x}) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} f_m(\mathbf{x}) & \frac{\partial}{\partial x_2} f_m(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_m(\mathbf{x}) \end{bmatrix}$$

For a small enough update $\mathbf{y} \in \mathbb{R}^n$, Taylor's theorem states

$$\mathbf{f}(\mathbf{x} + \mathbf{y}) \approx \mathbf{f}(\mathbf{x}) + \mathcal{J}\mathbf{f}(\mathbf{x}) \cdot \mathbf{y}$$

Jacobian descent

Goal: minimize $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ according to the $\preceq$ partial order. For any $\mathbf{x} \in \mathbb{R}^n$, the Jacobian of $\mathbf{f}$ at $\mathbf{x}$, is

$$\mathcal{J}\mathbf{f}(\mathbf{x}) = \begin{bmatrix} \nabla f_1(\mathbf{x})^\top \\ \nabla f_2(\mathbf{x})^\top \\ \vdots \\ \nabla f_m(\mathbf{x})^\top \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x_1} f_1(\mathbf{x}) & \frac{\partial}{\partial x_2} f_1(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_1(\mathbf{x}) \\ \frac{\partial}{\partial x_1} f_2(\mathbf{x}) & \frac{\partial}{\partial x_2} f_2(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_2(\mathbf{x}) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} f_m(\mathbf{x}) & \frac{\partial}{\partial x_2} f_m(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_m(\mathbf{x}) \end{bmatrix}$$

For a small enough update $\mathbf{y} \in \mathbb{R}^n$, Taylor's theorem states

$$\mathbf{f}(\mathbf{x} + \mathbf{y}) \approx \mathbf{f}(\mathbf{x}) + \mathcal{J}\mathbf{f}(\mathbf{x}) \cdot \mathbf{y}$$

We want $\mathbf{f}(\mathbf{x} + \mathbf{y}) \preceq \mathbf{f}(\mathbf{x})$.

Jacobian descent

Goal: minimize $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ according to the $\preceq$ partial order. For any $\mathbf{x} \in \mathbb{R}^n$, the Jacobian of $\mathbf{f}$ at $\mathbf{x}$, is

$$\mathcal{J}\mathbf{f}(\mathbf{x}) = \begin{bmatrix} \nabla f_1(\mathbf{x})^\top \\ \nabla f_2(\mathbf{x})^\top \\ \vdots \\ \nabla f_m(\mathbf{x})^\top \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x_1} f_1(\mathbf{x}) & \frac{\partial}{\partial x_2} f_1(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_1(\mathbf{x}) \\ \frac{\partial}{\partial x_1} f_2(\mathbf{x}) & \frac{\partial}{\partial x_2} f_2(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_2(\mathbf{x}) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} f_m(\mathbf{x}) & \frac{\partial}{\partial x_2} f_m(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_m(\mathbf{x}) \end{bmatrix}$$

For a small enough update $\mathbf{y} \in \mathbb{R}^n$, Taylor's theorem states

$$\mathbf{f}(\mathbf{x} + \mathbf{y}) \approx \mathbf{f}(\mathbf{x}) + \mathcal{J}\mathbf{f}(\mathbf{x}) \cdot \mathbf{y}$$

We want $\mathbf{f}(\mathbf{x} + \mathbf{y}) \preceq \mathbf{f}(\mathbf{x})$. We select $\mathbf{y} = -\eta \mathcal{A}(\mathcal{J}\mathbf{f}(\mathbf{x})) \in \mathbb{R}^n$.

Jacobian descent

The mapping $\mathcal{A} : \mathbb{R}^{m \times n} \rightarrow \mathbb{R}^n$ is called an **aggregator**, it parameterizes **Jacobian descent**:

Jacobian descent

The mapping $\mathcal{A} : \mathbb{R}^{m \times n} \rightarrow \mathbb{R}^n$ is called an **aggregator**, it parameterizes **Jacobian descent**:

Algorithm 1: Jacobian descent with aggregator $\mathcal{A}$

Input: $\mathbf{x} \in \mathbb{R}^n$, $0 < \eta$, $T \in \mathbb{N}$, $\mathcal{A} : \mathbb{R}^{m \times n} \rightarrow \mathbb{R}^n$

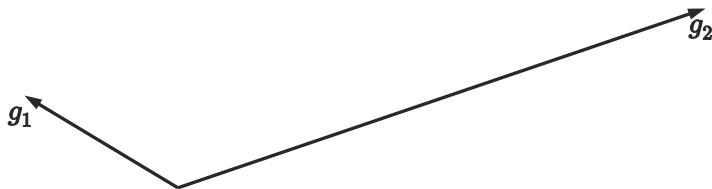
for $t \leftarrow 1$ **to** T **do**

$\mathbf{x} \leftarrow \mathbf{x} - \eta \mathcal{A}(\mathcal{J}\mathbf{f}(\mathbf{x}))$

Output: $\mathbf{x}$

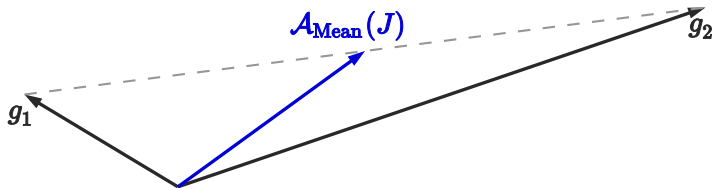
Unconflicting projection of gradients (UPGrad)

$$J = \begin{bmatrix} g_1^\top \\ g_2^\top \end{bmatrix}$$

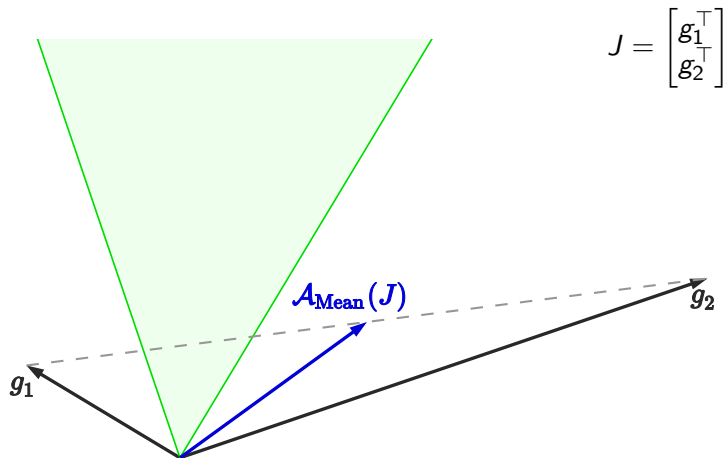


Unconflicting projection of gradients (UPGrad)

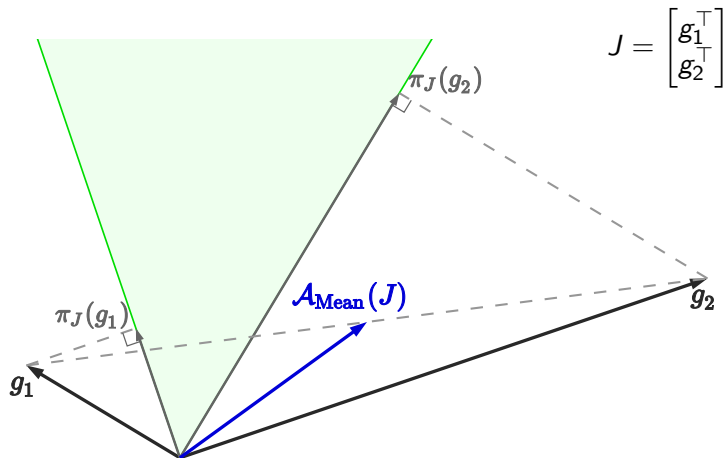
$$J = \begin{bmatrix} g_1^\top \\ g_2^\top \end{bmatrix}$$



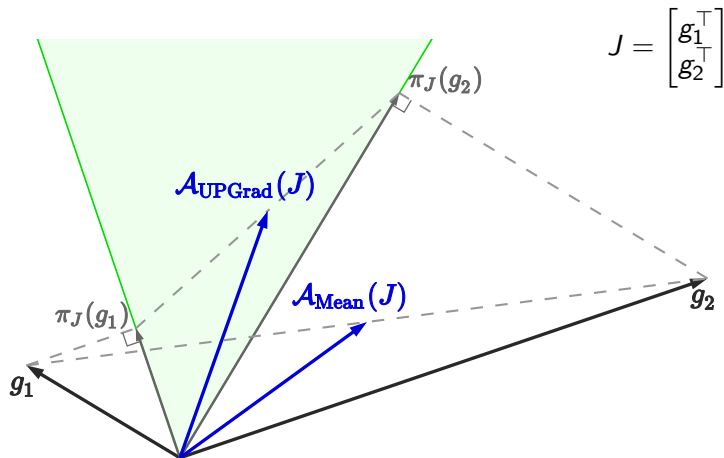
Unconflicting projection of gradients (UPGrad)



Unconflicting projection of gradients (UPGrad)



Unconflicting projection of gradients (UPGrad)



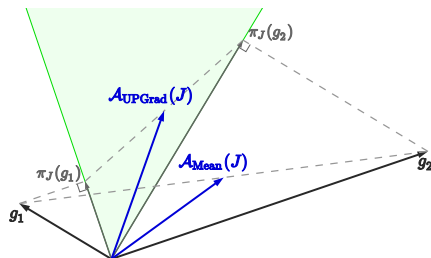
Formal definition of UPGrad

Given $J \in \mathbb{R}^{m \times n}$, let C_J be the cone $\{\mathbf{y} \in \mathbb{R}^n : \mathbf{0} \preceq J\mathbf{y}\}$. $\mathcal{A}$ is non-conflicting if for all $J \in \mathbb{R}^{m \times n}$, $\mathcal{A}(J) \in C_J$.

For any $\mathbf{x} \in \mathbb{R}^n$, the projection of $\mathbf{x}$ onto C_J is

$$\pi_J(\mathbf{x}) = \arg \min_{\mathbf{y} \in C_J} \|\mathbf{x} - \mathbf{y}\|^2$$

$$\mathcal{A}_{\text{UPGrad}}(J) = \frac{1}{m} \sum_{i \in [m]} \pi_J(g_i)$$



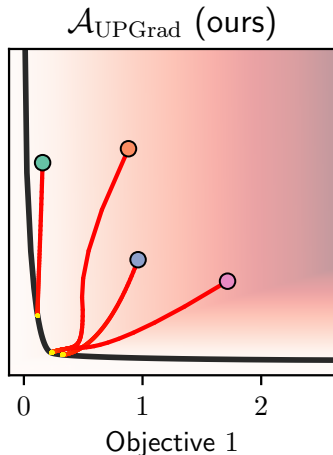
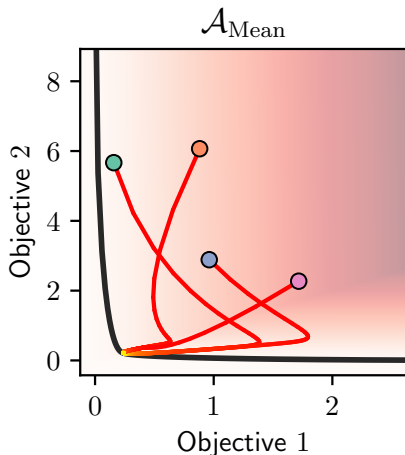
Applications and experimentation

Optimization trajectories

Goal: Minimize some convex quadratic function $f : \mathbb{R}^2 \rightarrow \mathbb{R}$, with conflicting gradients.

Optimization trajectories

Goal: Minimize some convex quadratic function $f : \mathbb{R}^2 \rightarrow \mathbb{R}^2$, with conflicting gradients.



Optimization trajectories: code

```
from torchjd import autojac
from torchjd.aggregation import UPGrad

x = tensor([0., 0.]) # initial value of x

for i in range(n_iters):
    y = f(x) # shape: [2]
    autojac.backward(y)
    # x now has a .jac field
    autojac.jac_to_grad([x], UPGrad())
    # x now has a .grad field
    optimizer.step()
    optimizer.zero_grad()
```

Full project available at github.com/TorchJD/trajectories

Per-sample JD

Each element of the batch has its own loss.

Per-sample JD

Each element of the batch has its own loss.

```
from torchjd import autojac
from torchjd.aggregation import UPGrad

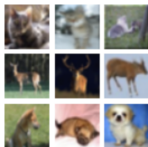
batch_size = 16
model = Linear(10, 1)

for x, y in dataloader:
    z = model(x).squeeze(dim=1) # shape: [16]
    losses = loss_fn(z, y) # shape: [16]
    autojac.backward(losses)
    # parameters now have a .jac field
    autojac.jac_to_grad(model.parameters(), UPGrad())
    # parameters now have a .grad field
    optimizer.step()
    optimizer.zero_grad()
```

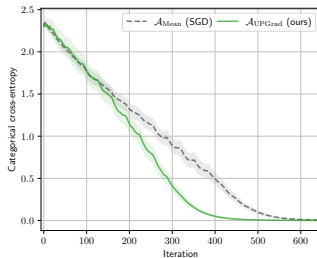
Full code example available at torchjd.org/latest/examples/iwrm.

Per-sample JD: results

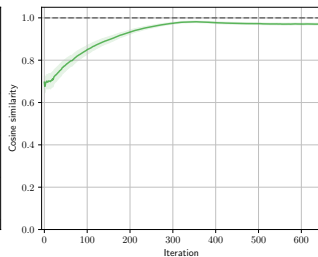
CIFAR-10



Training Loss



Update Similarity

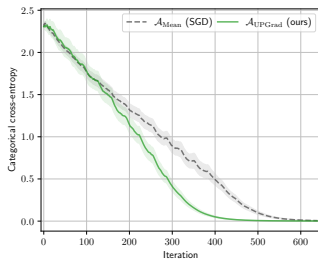


Per-sample JD: results

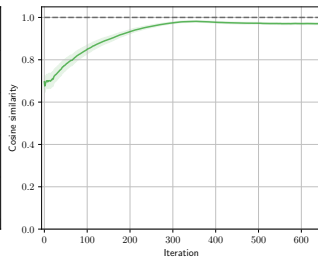
CIFAR-10



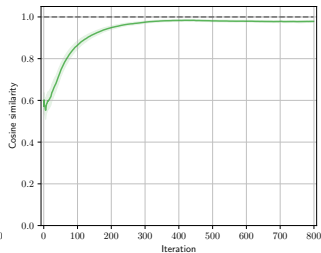
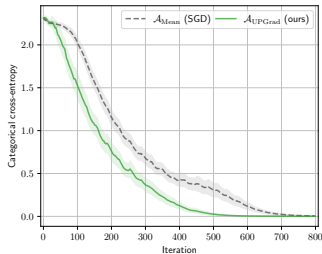
Training Loss



Update Similarity



SVHN

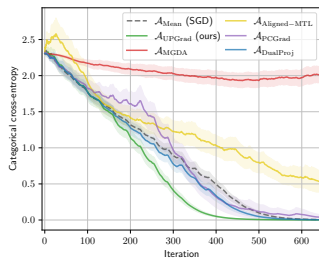


Per-sample JD: results

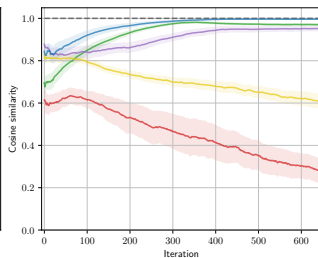
CIFAR-10



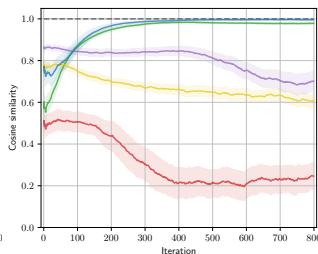
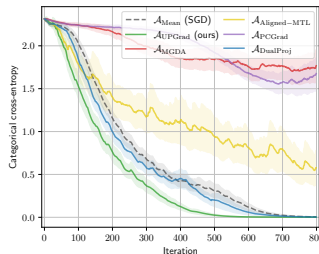
Training Loss



Update Similarity



SVHN

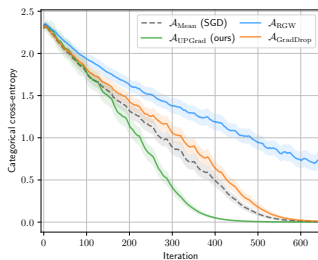


Per-sample JD: results

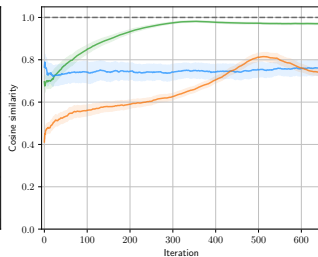
CIFAR-10



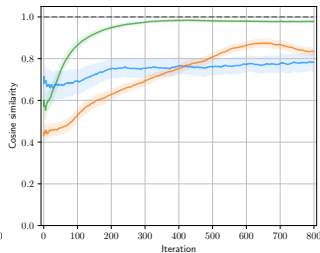
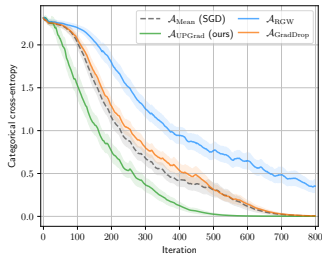
Training Loss



Update Similarity



SVHN

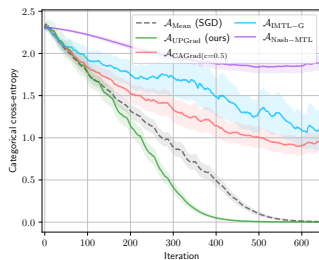


Per-sample JD: results

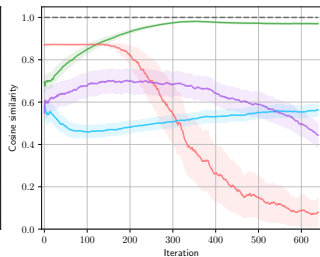
CIFAR-10



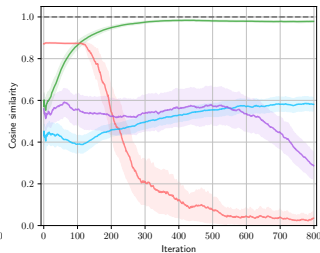
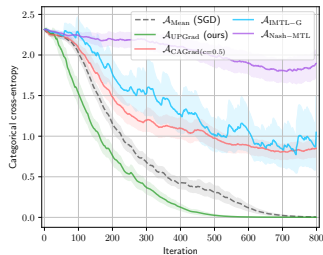
Training Loss



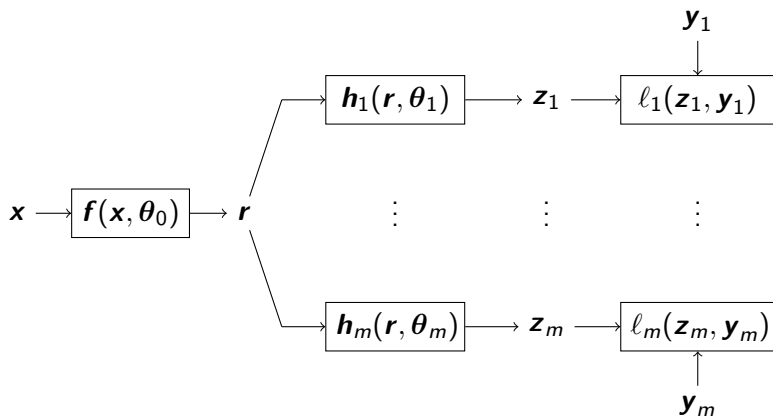
Update Similarity



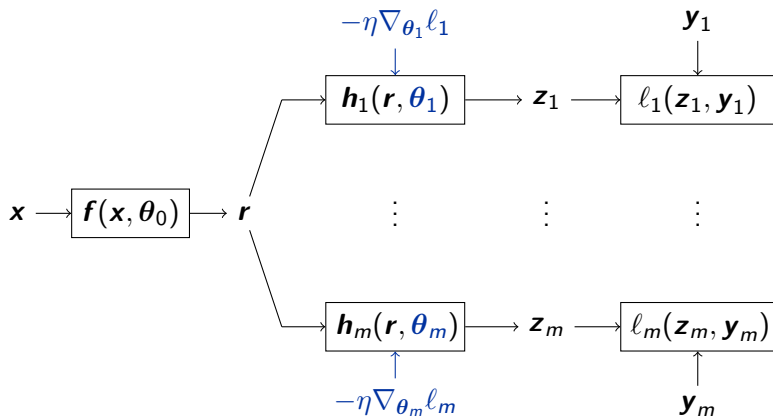
SVHN



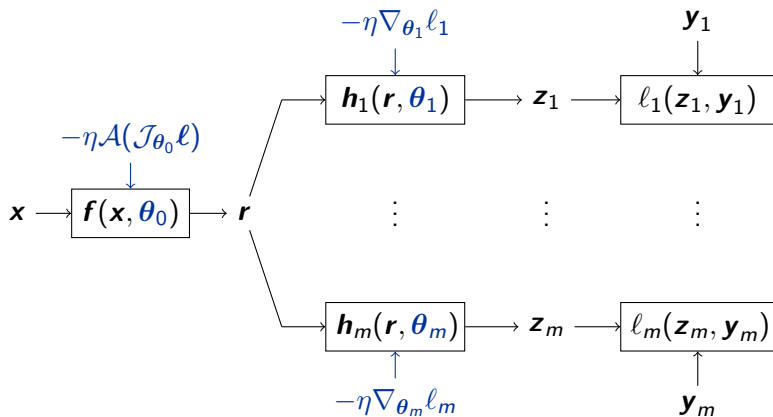
Multi-task learning



Multi-task learning



Multi-task learning



Multi-task learning: code

```
from torchjd import autojac
from torchjd.aggregation import UPGrad

f = Linear(10, 5)
h1 = Linear(5, 1)
h2 = Linear(5, 1)
for x, y1, y2 in dataloader:
    r = f(x)
    z1, z2 = h1(r), h2(r)
    l1, l2 = loss_fn(z1, y1), loss_fn(z2, y2)
    autojac.mtl_backward([l1, l2], features=r)
    # head parameters now have a .grad field
    # backbone parameters now have a .jac field
    autojac.jac_to_grad(f.parameters(), UPGrad())
    # All parameters now have a .grad field
    optimizer.step()
    optimizer.zero_grad()
```

Full code example available at torchjd.org/latest/examples/mtl.

Conclusion

Many problems are multi-objective.

Conclusion

Many problems are multi-objective.

Dual cone is rarely empty in deep learning.

Conclusion

Many problems are multi-objective.

Dual cone is rarely empty in deep learning.

TorchJD to compute and aggregate Jacobians.

Conclusion

Many problems are multi-objective.

Dual cone is rarely empty in deep learning.

TorchJD to compute and aggregate Jacobians.

Try to compute the inner products between your gradients.

Conclusion

Many problems are multi-objective.

Dual cone is rarely empty in deep learning.

TorchJD to compute and aggregate Jacobians.

Try to compute the inner products between your gradients.

Can JD solve problems that GD can't?

Appendix

Gramian-based JD

$\mathcal{A}_{\text{UPGrad}}(\mathcal{J}\mathbf{f}(\mathbf{x}))$ is a linear combination of the rows of $\mathcal{J}\mathbf{f}(\mathbf{x})$.

Gramian-based JD

$\mathcal{A}_{\text{UPGrad}}(\mathcal{J}\mathbf{f}(\mathbf{x}))$ is a linear combination of the rows of $\mathcal{J}\mathbf{f}(\mathbf{x})$.

The weights only depend on the (small) Gramian
 $\mathcal{G}\mathbf{f}(\mathbf{x}) = \mathcal{J}\mathbf{f}(\mathbf{x}) \cdot \mathcal{J}\mathbf{f}(\mathbf{x})^\top$.

Gramian-based JD

$\mathcal{A}_{\text{UPGrad}}(\mathcal{J}\mathbf{f}(\mathbf{x}))$ is a linear combination of the rows of $\mathcal{J}\mathbf{f}(\mathbf{x})$.

The weights only depend on the (small) Gramian
 $\mathcal{G}\mathbf{f}(\mathbf{x}) = \mathcal{J}\mathbf{f}(\mathbf{x}) \cdot \mathcal{J}\mathbf{f}(\mathbf{x})^\top$.

We can compute $\mathcal{G}\mathbf{f}(\mathbf{x})$ directly, extract the weights, combine the losses, and do a step of gradient descent.

Gramian-based JD

$\mathcal{A}_{\text{UPGrad}}(\mathcal{J}\mathbf{f}(\mathbf{x}))$ is a linear combination of the rows of $\mathcal{J}\mathbf{f}(\mathbf{x})$.

The weights only depend on the (small) Gramian
 $\mathcal{G}\mathbf{f}(\mathbf{x}) = \mathcal{J}\mathbf{f}(\mathbf{x}) \cdot \mathcal{J}\mathbf{f}(\mathbf{x})^\top$.

We can compute $\mathcal{G}\mathbf{f}(\mathbf{x})$ directly, extract the weights, combine the losses, and do a step of gradient descent.

We have developed the `autogram` engine to compute $\mathcal{G}\mathbf{f}(\mathbf{x})$ efficiently (work in progress).

Gramian-based JD

$\mathcal{A}_{\text{UPGrad}}(\mathcal{J}\mathbf{f}(\mathbf{x}))$ is a linear combination of the rows of $\mathcal{J}\mathbf{f}(\mathbf{x})$.

The weights only depend on the (small) Gramian
 $\mathcal{G}\mathbf{f}(\mathbf{x}) = \mathcal{J}\mathbf{f}(\mathbf{x}) \cdot \mathcal{J}\mathbf{f}(\mathbf{x})^\top$.

We can compute $\mathcal{G}\mathbf{f}(\mathbf{x})$ directly, extract the weights, combine the losses, and do a step of gradient descent.

We have developed the `autogram` engine to compute $\mathcal{G}\mathbf{f}(\mathbf{x})$ efficiently (work in progress).

This saves a lot of memory, so it's often much faster.

Gramian-based JD

$\mathcal{A}_{\text{UPGrad}}(\mathcal{J}\mathbf{f}(\mathbf{x}))$ is a linear combination of the rows of $\mathcal{J}\mathbf{f}(\mathbf{x})$.

The weights only depend on the (small) Gramian
 $\mathcal{G}\mathbf{f}(\mathbf{x}) = \mathcal{J}\mathbf{f}(\mathbf{x}) \cdot \mathcal{J}\mathbf{f}(\mathbf{x})^\top$.

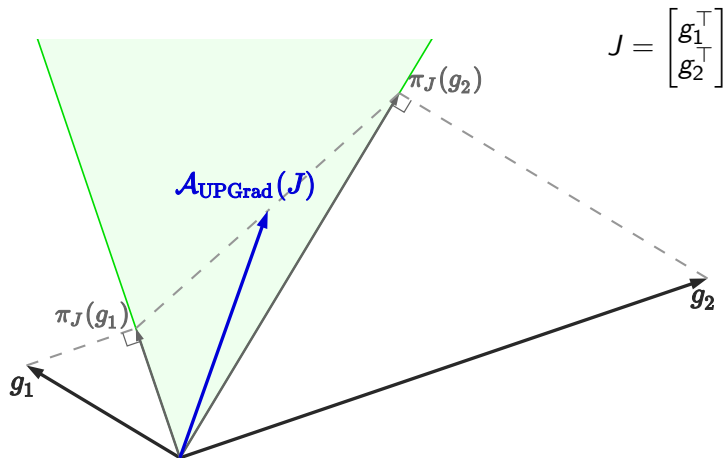
We can compute $\mathcal{G}\mathbf{f}(\mathbf{x})$ directly, extract the weights, combine the losses, and do a step of gradient descent.

We have developed the `autogram` engine to compute $\mathcal{G}\mathbf{f}(\mathbf{x})$ efficiently (work in progress).

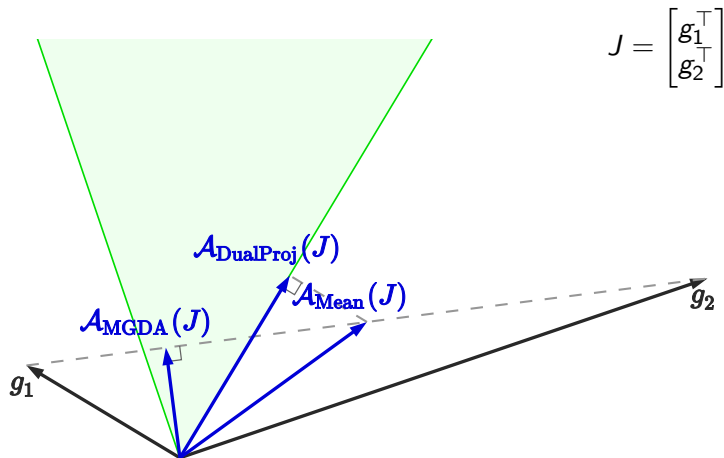
This saves a lot of memory, so it's often much faster.

Applicable to most other aggregators from the literature.

Other aggregators



Other aggregators

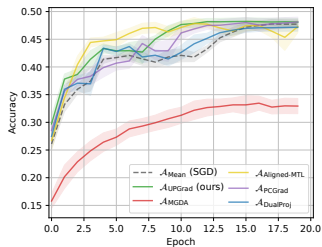


Per-sample JD: test accuracies

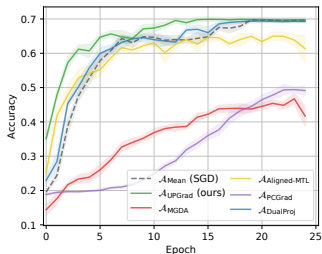
CIFAR-10



Test Accuracy

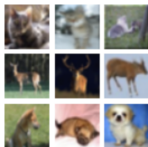


SVHN

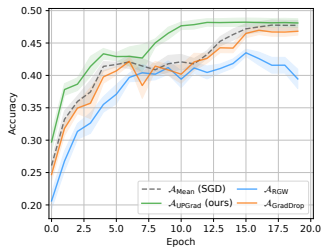


Per-sample JD: test accuracies

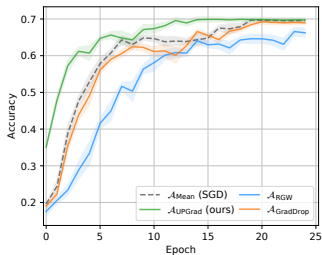
CIFAR-10



Test Accuracy

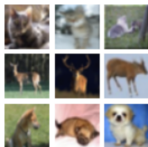


SVHN

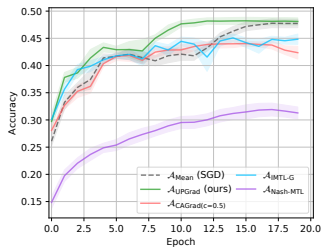


Per-sample JD: test accuracies

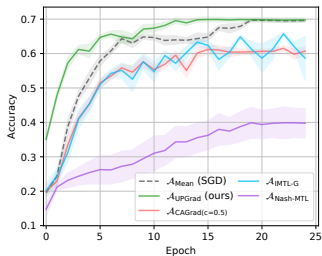
CIFAR-10



Test Accuracy

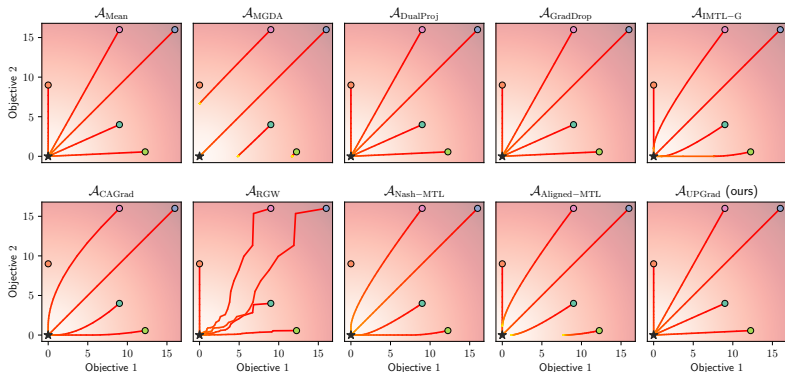


SVHN



Element-wise quadratic function trajectories

Minimize $\mathbf{f} : \begin{bmatrix} x \\ y \end{bmatrix} \mapsto \begin{bmatrix} x^2 \\ y^2 \end{bmatrix}$. It's Jacobian at $\begin{bmatrix} x \\ y \end{bmatrix}$ is $\begin{bmatrix} 2x & 0 \\ 0 & 2y \end{bmatrix}$.



Convex quadratic form trajectories

Some convex quadratic function $\mathbf{f} : \mathbb{R}^2 \rightarrow \mathbb{R}^2$, with conflicting gradients.

